

Blatt 05, Aufgabe 1.1

Wo gibt es Unterschiede, was sind die Gründe dafür? Welche der Varianten ist aufwändiger in der Implementierung?

Beim Regex- bzw. Scanner-Ansatz werden einzelne Ausdrücke direkt auf den Text angewendet. Dadurch könnten leichter Konflikte entstehen, beispielsweise wenn Keywords innerhalb von Kommentaren oder Strings vorkommen.

Bei ANTLR wird der Text vom Lexer in Tokens zerlegt. Befindet sich ein Wort in einem String, ist es direkt Teil eines String-Tokens und wird nicht dazu als Keyword erkannt. Das passiert, weil Regex nach Textmustern sucht, während ANTLR den Quelltext anhand einer Grammatik analysiert und in sprachliche Bestandteile zerlegt.

Die ANTLR-Variante wäre insgesamt aufwändiger zu implementieren, wenn die Grammatik selbst erstellt werden müsste. Da die Grammatik in dieser Aufgabe bereits vorgegeben war, ist die Implementierung des Syntaxhighlightings vergleichsweise wenig aufwändig.

Aufgabe 1.2

```

> Task :highlighting.antlr.PrettyPrinterDemo.main()
Einrückung in Leerzeichen: 4
===== Beispiel =====
public class Einfach{
    private String name;
    public void test(){
        return;
    }
}

===== Beispiel =====
public class IfBedingung{
    private String text;
    public void test(){
        if(text!=null){
            return;
        }
        else{
            while(text!=null){
                text=null;
            }
        }
    }
}

===== Beispiel =====
public class Verschachtelt{
    public void test(){
        {
            {
                return;
            }
        }
    }
}

```

Aufgabe 2.1: Äquivalenzklassenbildung und Grenzwertanalyse für `Shop.accept`

Analyse der Anforderungen

Ein Auftrag wird nur angenommen, wenn alle folgenden Bedingungen erfüllt sind:

1. Das Fahrrad ist kein Gravel-Bike.
2. Das Fahrrad ist kein E-Bike.
3. Der Kunde besitzt aktuell keinen weiteren offenen Auftrag.
4. Es befinden sich höchstens vier andere offene Aufträge in der Warteschlange.

Der Rückgabewert ist `true` , wenn der Auftrag angenommen und in die Warteschlange aufgenommen wird, sonst `false`.

Äquivalenzklassen

Fahrradtyp

Äquivalenzklasse	Beschreibung	Erwartetes Ergebnis
EK1	Fahrradtyp = RACE, SINGLE_SPEED oder FIXIE	Auftrag wird angenommen
EK2	Fahrradtyp = GRAVEL	Auftrag wird abgelehnt
EK3	Fahrradtyp = EBIKE	Auftrag wird abgelehnt

Vorhandene offene Aufträge des Kunden

Äquivalenzklasse	Beschreibung	Erwartetes Ergebnis
EK4	Kunde besitzt keinen offenen Auftrag	Auftrag kann angenommen werden
EK5	Kunde besitzt bereits einen offenen Auftrag	Auftrag wird abgelehnt

Anzahl offener Aufträge im Shop

Äquivalenzklasse	Beschreibung	Erwartetes Ergebnis
EK6	0 bis 4 offene Aufträge vorhanden	Auftrag kann angenommen werden
EK7	5 oder mehr offene Aufträge vorhanden	Auftrag wird abgelehnt

Grenzwertanalyse

Die maximale Anzahl offener Aufträge beträgt fünf. Relevant sind daher die Werte unmittelbar am Grenzwert.

Anzahl offener Aufträge vor Aufruf von <code>accept</code>	Erwartetes Ergebnis
3	Auftrag wird angenommen
4	Auftrag wird angenommen
5	Auftrag wird abgelehnt

Der entscheidende Grenzwert liegt zwischen 4 und 5 offenen Aufträgen.

Abgeleitete Testfälle

Testfall	Fahrradtyp	Kunde hat offenen Auftrag	Anzahl offener Aufträge	Erwartetes Ergebnis
TF1	RACE	Nein	0	true
TF2	GRAVEL	Nein	0	false
TF3	EBIKE	Nein	0	false
TF4	RACE	Ja	1	false
TF5	RACE	Nein	4	true
TF6	RACE	Nein	5	false

Diese Testfälle decken alle relevanten Äquivalenzklassen sowie den kritischen Grenzwert der maximalen Warteschlangengröße ab.

Aufgabe 2.2: Mocking mit Mockito

Einsatz von Mockito

Die Klasse `Order` ist in der Implementierung nicht vollständig umgesetzt. Die Methoden

```
getBicycleType()
```

und

```
getCustomer()
```

werfen jeweils eine `UnsupportedOperationException` .

Deshalb können keine `Order` -Objekte für die Tests verwendet werden.

--> Mock-Objekte mit Mockito:

```
Order order = mock(Order.class);

when(order.getBicycleType()).thenReturn(Type.RACE);
when(order.getCustomer()).thenReturn("Alice");
```

Die gemockten Objekte simulieren das Verhalten einer vollständigen Implementierung von `Order` . Dadurch können gezielt verschiedene Fahrradtypen und Kundenkonstellationen erzeugt werden.

Begründung der Anwendung von Mockito

Mockito wird verwendet, um die noch nicht implementierte Klasse `Order` zu simulieren. Dadurch können die Rückgabewerte für `getBicycleType()` und `getCustomer()` festgelegt werden, ohne die eigentliche Klasse implementieren zu müssen.

Die zu testende Methode `Shop.accept()` wird dabei nicht gemockt, sondern vollständig real ausgeführt. Mockito wird nur für die `Order` -Objekte verwendet.

Dadurch kann das Verhalten von `Shop.accept()` unabhängig von der unvollständigen Implementierung der Klasse `Order` getestet werden.

Testergebnis

Ergebnis:

```
✓ 6 tests passed 6 tests total, 405 ms

> Task :compileJava UP-TO-DATE
> Task :processResources NO-SOURCE
> Task :classes UP-TO-DATE
> Task :compileTestJava UP-TO-DATE
> Task :processTestResources NO-SOURCE
> Task :testClasses UP-TO-DATE
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
> Task :test
BUILD SUCCESSFUL in 2s
3 actionable tasks: 1 executed, 2 up-to-date
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.5.1/userguide/configuration\_cache\_enabling.html
11:14:01: Execution finished 'test'.
```